

DP 스터디

2026년 4월 6일

증가하는 부분 수열의 개수(22971)

문제

길이가 N ($1 \leq N \leq 5\,000$)인 수열 A 가 주어진다. 수열의 i 번째 원소 A_i ($1 \leq A_i \leq 5\,000$)로 끝나는 증가하는 부분 수열의 개수를 출력하는 프로그램을 작성하자.

단, 수가 너무 커질 수 있으니 998 244 353으로 나눈 나머지를 출력한다.

부분 수열이란 주어진 수열에서 1개 이상의 원소를 골라 원래 순서대로 나열하여 얻은 수열을 말한다.

증가하는 부분 수열이란 맨 처음 원소를 제외한 모든 원소가 바로 전 원소보다 큰 수열을 말한다. 다시 말해 길이가 N 인 부분 수열 A 가 있을 때, $A_{i-1} < A_i$ ($2 \leq i \leq N$)를 만족하면 A 는 증가하는 부분 수열이다.

길이가 1인 부분 수열은 항상 증가하는 부분 수열임에 주의하자.

예제 입력/출력 1

입력

5
1 2 3 4 5

출력

1 2 4 8 16

예제 입력/출력 3

입력

5
1 2 2 4 3

출력

1 2 2 6 6

김시온 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5      static final int mod = 998244353;
6
7      static int n;
8      static int[] a, c;
9
10     // dp(i) = a_i에서 끝나는 LIS의 개수
11     static int dp(int i) {
12         if (c[i] > 0) {
13             return c[i];
14         }
15         int sum = 1;
16         for (int j = 0; j < i; j++) {
17             if (a[j] < a[i]) {
18                 sum = (sum + dp(j)) % mod;
19             }
20         }
21         return c[i] = sum;
22     }
23
24     public static void main(String[] args) throws IOException {
25         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
26         StringBuilder bw = new StringBuilder();
27         n = Integer.parseInt(br.readLine());
28         a = new int[n];
29         StringTokenizer st = new StringTokenizer(br.readLine());
```

김시온 소스 코드 2

```
30     for (int i = 0; i < n; i++) {
31         a[i] = Integer.parseInt(st.nextToken());
32     }
33     c = new int[n];
34     for (int i = 0; i < n; i++) {
35         bw.append(dp(i));
36         if (i == n - 1) {
37             bw.append('\n');
38         } else {
39             bw.append(' ');
40         }
41     }
42     System.out.print(bw);
43 }
44
45 }
```

원찬혁 소스 코드 1

```
1 package algo_test;
2
3 import java.io.*;
4 import java.util.*;
5
6 public class Main {
7     static int ary[], dp[];
8     public static void main(String[] args) throws Exception {
9         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
10        StringBuilder sb = new StringBuilder();
11        int n = Integer.parseInt(br.readLine());
12        StringTokenizer st = new StringTokenizer(br.readLine());
13        ary = new int[n];
14        dp = new int[n];
15        for(int i = 0; i < n; i++)
16            ary[i] = Integer.parseInt(st.nextToken());
17        dp[0] = 1;
18        sb.append(1).append(" ");
19        for(int i = 1; i < n; i++) {
20            dp[i] = 1;
21            for(int j = 0; j < i; j++) {
22                if(ary[j] < ary[i])
23                    dp[i] += dp[j];
24            }
25            dp[i] %= 998244353;
26            sb.append(dp[i]).append(" ");
27        }
28        System.out.println(sb);
```

원찬혁 소스 코드 2

```
29     }  
30 }
```


손현준 소스 코드 1

```
1 package baekjoon;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.util.StringTokenizer;
7
8 public class P22971 {
9     public static void main(String[] args) throws IOException {
10         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
11         StringBuilder sb = new StringBuilder();
12         StringTokenizer st;
13
14         final int MOD = 998244353;
15
16         int N = Integer.parseInt(br.readLine());
17         int[] num = new int[N];
18         long[] dp = new long[N];
19
20         st = new StringTokenizer(br.readLine(), " ");
21         for (int n = 0; n < N; n++) {
22             num[n] = Integer.parseInt(st.nextToken());
23         }
24
25         for (int n = 0; n < N; n++) {
26             int tNum = num[n];
27             long sum = 1;
28
29             for (int i = n - 1; i >= 0; i--) {
```

손현준 소스 코드 2

```
30         if (num[i] < tNum) sum = (sum + dp[i]) % MOD;
31     }
32
33     dp[n] = sum;
34     sb.append(sum).append(" ");
35 }
36 System.out.println(sb.toString());
37 }
38 }
```

김지훈 소스 코드 1

```
1  import java.io.BufferedReader;
2  import java.io.InputStreamReader;
3  import java.util.Arrays;
4  import java.util.StringTokenizer;
5
6  public class BOJ_22971 {
7      static int N;
8      static int[] A;
9      static long[] memo;
10     static final int MOD = 998244353;
11
12     public static void main(String[] args) throws Exception {
13         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
14
15         N = Integer.parseInt(br.readLine());
16         A = new int[N];
17         memo = new long[N];
18         Arrays.fill(memo, -1);
19
20         StringTokenizer st = new StringTokenizer(br.readLine());
21         for (int i = 0; i < N; i++) {
22             A[i] = Integer.parseInt(st.nextToken());
23         }
24
25         StringBuilder sb = new StringBuilder();
26         for (int i = 0; i < N; i++) {
27             sb.append(solve(i)).append(" ");
28         }
29         System.out.println(sb);
30     }
31 }
```

김지훈 소스 코드 2

```
30     }
31
32     private static long solve(int i) {
33
34         if (memo[i] != -1)
35             return memo[i];
36
37         long count = 1;
38         for (int j = 0; j < i; j++) {
39             if (A[j] < A[i]) {
40                 count = (count + solve(j)) % MOD;
41             }
42         }
43
44         return memo[i] = count;
45     }
46 }
```

김준형 소스 코드 1

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Main {
5      static int n;
6      static long[] dp;
7      static int[] arr;
8      static final int DIV = 998244353;
9      public static void main(String[] args) throws IOException {
10         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
11         StringBuilder sb = new StringBuilder();
12         n = Integer.parseInt(br.readLine());
13         dp = new long[n+1];
14         Arrays.fill(dp, 1);
15         arr = new int[n];
16         StringTokenizer st = new StringTokenizer(br.readLine());
17
18         for(int i=0; i<n; i++) {
19             arr[i] = Integer.parseInt(st.nextToken());
20         }
21
22         for(int i=0; i<n; i++) {
23             int temp = arr[i];
24             for(int j=0; j<i; j++) {
25                 if(arr[j]<temp) {
26                     dp[i]=(dp[i]+dp[j])%DIV;
27                 }
28             }
29             sb.append(dp[i]%998244353).append(" ");
30         }
31     }
32 }
```

김준형 소스 코드 2

```
30         }  
31  
32         System.out.print(sb);  
33     }  
34  
35 }
```

서민종 소스 코드 1

```
1 package BOJ22971;
2
3 import java.util.*;
4 import java.io.*;
5
6 public class Main {
7     static long INF = 998244353;
8     static int N;
9     static int[] numbers;
10    public static void main(String[] args) throws IOException {
11        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
12        BufferedWriter bw = new BufferedWriter(new
13            ↳ OutputStreamWriter(System.out));
14        N = Integer.parseInt(br.readLine());
15        numbers = new int[N];
16        StringTokenizer st = new StringTokenizer(br.readLine());
17        for(int i=0; i<N; i++) {
18            numbers[i] = Integer.parseInt(st.nextToken());
19        }
20        long[] dp = new long[N];
21        dp[0] = 1;
22        for(int i=1; i<N; i++) {
23            long sum = 0;
24            for(int j=0; j<i; j++) {
25                if(numbers[j] < numbers[i]) {
26                    sum += dp[j];
27                }
28            }
29            dp[i] = (sum + 1) % INF;
```

서민종 소스 코드 2

```
29     }
30     StringBuilder sb = new StringBuilder();
31     for(int i=0; i<N; i++) {
32         sb.append(dp[i]).append(" ");
33     }
34     bw.write(sb.toString());
35     bw.flush();
36 }
37 }
38
```


임건애 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5
6      static int N;
7      static int[] arr, memo;
8      static final int MOD = 998244353;
9
10     public static void main(String[] args) throws IOException {
11         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
12
13         N = Integer.parseInt(br.readLine());
14
15         StringTokenizer st = new StringTokenizer(br.readLine());
16         arr = new int[N];
17         for (int i = 0; i < N; i++) {
18             arr[i] = Integer.parseInt(st.nextToken());
19         }
20
21         memo = new int[N];
22         Arrays.fill(memo, -1);
23
24         StringBuilder sb = new StringBuilder();
25
26         for (int i = 0; i < N; i++) {
27             sb.append(solve(i)).append(" ");
28         }
29     }
```

임건애 소스 코드 2

```
30         System.out.println(sb);
31     }
32
33     static int solve(int idx) {
34         if (memo[idx] != -1) return memo[idx];
35
36         int count = 1;
37         for (int i = 0; i < idx; i++) {
38             if (arr[i] < arr[idx]) {
39                 count = (count + solve(i)) % MOD;
40             }
41         }
42
43         return memo[idx] = count;
44     }
45
46 }
```

임건애 소스 코드 (BottomUp) 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5
6      static final int temp = 998244353;
7
8      public static void main(String[] args) throws IOException {
9          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
10
11          int N = Integer.parseInt(br.readLine());
12
13          int[] arr = new int[N];
14          StringTokenizer st = new StringTokenizer(br.readLine());
15          for (int i = 0; i < N; i++) {
16              arr[i] = Integer.parseInt(st.nextToken());
17          }
18
19          StringBuilder sb = new StringBuilder();
20          int[] dp = new int[N];
21          dp[0] = 1;
22          sb.append(dp[0]).append(" ");
23          for (int i = 1; i < N; i++) {
24              dp[i] = 1;
25              for (int j = 0; j < i; j++) {
26                  if (arr[i] > arr[j]) {
27                      dp[i] = (dp[i] + dp[j]) % temp;
28                  }
29              }
30          }
31      }
32  }
```

임건애 소스 코드 (BottomUp) 2

```
29         }
30         sb.append(dp[i]).append(" ");
31     }
32
33     System.out.println(sb);
34 }
35
36 }
```

징검다리 건너기(21317)

문제

심마니 영재는 산삼을 찾아다닌다.

산삼을 찾던 영재는 $N(1 \leq N \leq 20)$ 개의 돌이 일렬로 나열되어 있는 강가를 발견했고, 마지막 돌 틈 사이에 산삼이 있다는 사실을 알게 되었다.

마지막 돌 틈 사이에 있는 산삼을 캐기 위해 영재는 돌과 돌 사이를 점프하면서 이동하며, 점프의 종류는 3가지가 있다. 현재 위치에서 다음 돌로 이동하는 작은 점프, 1개의 돌을 건너뛰어 이동하는 큰 점프, 2개의 돌을 건너뛰어 이동하는 매우 큰 점프가 있다.

각 점프를 할 때는 에너지를 소비하는데, 작은 점프와 큰 점프 시 소비되는 에너지는 점프를 하는 돌의 번호마다 다르다. 매우 큰 점프는 단 한 번의 기회가 주어지며, 이때는 점프를 하는 돌의 번호와 상관없이 $K(1 \leq K \leq 5\,000)$ 만큼의 에너지를 소비한다. 작은 점프와 큰 점프 시 필요한 에너지도 5 000을 넘지 않는 자연수이다.

에너지를 최대한 아껴야 하는 영재가 첫 번째 돌에서 출발하여 산삼을 얻기 위해 필요한 최소 에너지를 구하여라.

예제 입력/출력 1

입력

5

1 2

2 3

4 5

6 7

4

출력

5

김시온 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5      static int n, k;
6      static int[] a, b;
7      static int[][] c;
8
9      // dp(i, j) = (i, i + 1)번째 돌 사이를 뛰어 넘으려고 한다. j는 매우 큰 점프를 했는지 여부.
10     static int dp(int i, int j) {
11         if (i == n - 1) {
12             return 0;
13         }
14         if (c[j][i] != -1) {
15             return c[j][i];
16         }
17         // 1. 작은 점프
18         int min = a[i] + dp(i + 1, j);
19         // 2. 큰 점프
20         if (i + 2 < n) {
21             min = Math.min(min, b[i] + dp(i + 2, j));
22         }
23         // 3. 매우 큰 점프
24         if (i + 3 < n && j == 0) {
25             min = Math.min(min, k + dp(i + 3, 1));
26         }
27         return c[j][i] = min;
28     }
29 }
```

김시온 소스 코드 2

```
30     public static void main(String[] args) throws IOException {
31         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
32         n = Integer.parseInt(br.readLine());
33         a = new int[n];
34         b = new int[n];
35         c = new int[2][n];
36         Arrays.fill(c[0], -1);
37         Arrays.fill(c[1], -1);
38
39         for (int i = 0; i < n - 1; i++) {
40             StringTokenizer st = new StringTokenizer(br.readLine());
41             a[i] = Integer.parseInt(st.nextToken());
42             b[i] = Integer.parseInt(st.nextToken());
43         }
44         k = Integer.parseInt(br.readLine());
45         System.out.println(dp(0, 0));
46     }
47 }
```


원찬혁 소스 코드 1

```
1 package algo_test;
2
3 import java.io.*;
4 import java.util.*;
5
6 public class Main {
7     static int miniJump[], bigJump[], dp[][], cheatJump;
8     public static void main(String[] args) throws Exception {
9         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
10        StringTokenizer st;
11        int n = Integer.parseInt(br.readLine());
12        miniJump = new int[n];
13        bigJump = new int[n];
14        dp = new int[n][2];
15        for(int i = 0; i < n - 1; i++) {
16            st = new StringTokenizer(br.readLine());
17            miniJump[i] = Integer.parseInt(st.nextToken());
18            bigJump[i] = Integer.parseInt(st.nextToken());
19        }
20        cheatJump = Integer.parseInt(br.readLine());
21        System.out.println(Math.min(sol(n - 1, 1), sol(n - 1, 0)));
22    }
23
24    static int sol(int idx, int m) {
25        if(idx <= 0)
26            return 0;
27        if(dp[idx][m] > 0)
28            return dp[idx][m];
29        dp[idx][m] = 1000000000;
```

원찬혁 소스 코드 2

```
30     if(m > 0) {
31         if(idx >= 3)
32             dp[idx][m] = sol(idx - 3, 0) + cheatJump;
33     }
34
35     dp[idx][m] = Math.min(dp[idx][m], sol(idx - 1, m) + miniJump[idx - 1]);
36     if(idx > 1)
37         dp[idx][m] = Math.min(dp[idx][m], sol(idx - 2, m) + bigJump[idx - 2]);
38
39     return dp[idx][m];
40 }
41 }
```

손현준 소스 코드 1

```
1  package baekjoon;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.util.StringTokenizer;
7
8  public class P21317 {
9      static class JumpCost {
10         int jumpS, jumpL;
11
12         public JumpCost(int jumpS, int jumpL) {
13             this.jumpS = jumpS;
14             this.jumpL = jumpL;
15         }
16     }
17
18     public static void main(String[] args) throws IOException {
19         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
20         StringTokenizer st;
21
22         int N = Integer.parseInt(br.readLine());
23         JumpCost[] jumpCost = new JumpCost[N - 1];
24         for (int n = 0; n < (N - 1); n++) {
25             st = new StringTokenizer(br.readLine(), " ");
26             int jumpS = Integer.parseInt(st.nextToken());
27             int jumpL = Integer.parseInt(st.nextToken());
28             jumpCost[n] = new JumpCost(jumpS, jumpL);
29         }
```

손현준 소스 코드 2

```
30     int K = Integer.parseInt(br.readLine());
31
32     // dp[바위 번호][매우 큰 점프 사용 여부]
33     int[][] dp = new int[N][2];
34
35     for (int n = 0; n < (N - 1); n++) {
36         int nCost = 0;
37
38         // 작은 점프 - K 사용 안한 상태
39         nCost = dp[n][0] + jumpCost[n].jumpS;
40         dp[n + 1][0] = (dp[n + 1][0] == 0) ? nCost : Integer.min(dp[n + 1][0],
41             ↪ nCost);
42         // 작은 점프 - K 사용한 상태
43         if (dp[n][1] != 0) {
44             nCost = dp[n][1] + jumpCost[n].jumpS;
45             dp[n + 1][1] = (dp[n + 1][1] == 0) ? nCost : Integer.min(dp[n +
46                 ↪ 1][1], nCost);
47         }
48
49         // 큰 점프 - K 사용 안한 상태
50         if (n == (N - 2)) continue;
51         nCost = dp[n][0] + jumpCost[n].jumpL;
52         dp[n + 2][0] = (dp[n + 2][0] == 0) ? nCost : Integer.min(dp[n + 2][0],
53             ↪ nCost);
54         // 큰 점프 - K 사용한 상태
55         if (dp[n][1] != 0) {
56             nCost = dp[n][1] + jumpCost[n].jumpL;
57             dp[n + 2][1] = (dp[n + 2][1] == 0) ? nCost : Integer.min(dp[n +
58                 ↪ 2][1], nCost);
59         }
60     }
```

손현준 소스 코드 3

```
55         }
56
57         // 매우 큰 점프
58         if (n == (N - 3)) continue;
59         nCost = dp[n][0] + K;
60         dp[n + 3][1] = (dp[n + 3][1] == 0) ? nCost : Integer.min(dp[n + 3][1],
        ↪ nCost);
61     }
62
63     System.out.println((dp[N - 1][1] == 0) ? dp[N - 1][0] : Integer.min(dp[N
        ↪ - 1][0], dp[N - 1][1]));
64 }
65 }
```

김준형 소스 코드 1

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Main {
5      static class Jump{
6          int s;
7          int m;
8          public Jump(int s, int m) {
9              this.s = s;
10             this.m = m;
11         }
12     }
13     static int n,k;
14     static Jump[] jumpArr;
15     static int dp[][];
16     public static void main(String[] args) throws IOException {
17         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
18         StringTokenizer st;
19         n = Integer.parseInt(br.readLine());
20         dp = new int[n+1][2];
21         for(int i=0; i<n+1; i++) {
22             Arrays.fill(dp[i], 1000000);
23         }
24         jumpArr = new Jump[n-1];
25
26         for(int i=0; i<n-1; i++) {
27             st = new StringTokenizer(br.readLine());
28             jumpArr[i] = new
                ↪ Jump(Integer.parseInt(st.nextToken()),Integer.parseInt(st.nextToken()));
```

김준형 소스 코드 2

```
29     }
30     k = Integer.parseInt(br.readLine());
31
32     dp[1][0] = 0; dp[1][1] = 0;
33     for(int i=1; i<=n; i++) {
34         // 슈퍼 점프 사용 안했을 때
35         if(i+1<=n) {
36             dp[i+1][0] = Math.min(dp[i+1][0], dp[i][0]+jumpArr[i-1].s);
37         }
38         if(i+2<=n) {
39             dp[i+2][0] = Math.min(dp[i+2][0], dp[i][0]+jumpArr[i-1].m);
40         }
41         if(i+3<=n) {
42             dp[i+3][1] = Math.min(dp[i+3][1], dp[i][0]+k);
43         }
44
45         // 슈퍼 점프 사용 했을 때
46         if(i+1<=n && i+1>3) {
47             dp[i+1][1] = Math.min(dp[i+1][1], dp[i][1]+jumpArr[i-1].s);
48         }
49         if(i+2<=n && i+2>3) {
50             dp[i+2][1] = Math.min(dp[i+2][1], dp[i][1]+jumpArr[i-1].m);
51         }
52     }
53
54     System.out.print(Math.min(dp[n][0], dp[n][1]));
55 }
```

김준형 소스 코드 3

```
56  
57 }
```


서민종 소스 코드 1

```
1 package BOJ21317;
2
3 import java.util.*;
4 import java.io.*;
5
6 public class Main {
7     static int N, K;
8     static int[][] jump;
9     public static void main(String[] args) throws IOException {
10         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
11         BufferedWriter bw = new BufferedWriter(new
            ↳ OutputStreamWriter(System.out));
12         N = Integer.parseInt(br.readLine());
13         jump = new int[N][2];
14         for(int i=1; i<=N-1; i++) {
15             StringTokenizer st = new StringTokenizer(br.readLine());
16             jump[i][0] = Integer.parseInt(st.nextToken());
17             jump[i][1] = Integer.parseInt(st.nextToken());
18         }
19         K = Integer.parseInt(br.readLine());
20         int[][] dp = new int[2][N+1];
21         Arrays.fill(dp[0], 1000000);
22         Arrays.fill(dp[1], 1000000);
23         dp[0][1] = 0;
24         if(N >= 2) {
25             dp[0][2] = jump[1][0];
26         }
27         if(N >= 3) {
28             dp[0][3] = Math.min(dp[0][1] + jump[1][1], dp[0][2] + jump[2][0]);
```

서민종 소스 코드 2

```
29     }
30     for(int i=3; i<=N; i++) {
31         dp[0][i] = Math.min(dp[0][i-2] + jump[i-2][1], dp[0][i-1] +
           ↪ jump[i-1][0]);
32         dp[1][i] = Math.min(dp[1][i-2] + jump[i-2][1], dp[1][i-1] +
           ↪ jump[i-1][0]);
33         dp[1][i] = Math.min(dp[1][i], dp[0][i-3] + K);
34     }
35     int answer = Math.min(dp[0][N], dp[1][N]);
36     bw.write(answer + "\n");
37     bw.flush();
38 }
39 }
```

임건애 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5
6      public static void main(String[] args) throws IOException {
7          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
8          StringTokenizer st;
9
10         int N = Integer.parseInt(br.readLine());
11
12         int[] small = new int[N + 1];
13         int[] big = new int[N + 1];
14         for (int i = 1; i < N; i++) {
15             st = new StringTokenizer(br.readLine());
16
17             small[i] = Integer.parseInt(st.nextToken());
18             big[i] = Integer.parseInt(st.nextToken());
19         }
20
21         int k = Integer.parseInt(br.readLine());
22
23         int[][] dp = new int[N + 1][2];
24         for (int i = 0; i <= N; i++) {
25             Arrays.fill(dp[i], 100001);
26         }
27
28         dp[1][0] = 0;
29         dp[1][1] = 0;
```

임건애 소스 코드 2

```
30
31     for (int i = 2; i <= N; i++) {
32         dp[i][0] = dp[i - 1][0] + small[i - 1];
33         dp[i][1] = dp[i - 1][1] + small[i - 1];
34
35         if (i >= 3) {
36             dp[i][0] = Math.min(dp[i][0], dp[i - 2][0] + big[i - 2]);
37             dp[i][1] = Math.min(dp[i][1], dp[i - 2][1] + big[i - 2]);
38         }
39
40         if (i >= 4) {
41             dp[i][1] = Math.min(dp[i][1], dp[i - 3][0] + k);
42         }
43     }
44
45     System.out.println(Math.min(dp[N][0], dp[N][1]));
46 }
47
48 }
```

사수빈탕(14585)

문제

수빈이는 좌표평면 위의 원점 $(0, 0)$ 에 앉아 있다. 좌표평면에는 N ($0 \leq N \leq 300$)개의 사탕 바구니가 있고, 각 사탕 바구니에는 처음에 M ($1 \leq M \leq 1\,000\,000$)개의 사탕이 들어 있다.

각 사탕 바구니는 $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ ($0 \leq x_i, y_i \leq 300$)에 있고, 위치는 서로 중복되지 않으며 $(0, 0)$ 에는 사탕이 없다.

시간이 1만큼 지날 때마다 사탕이 남아 있는 모든 사탕 바구니에서 사탕은 한 개씩 녹아서 없어진다. 수빈이는 사탕 바구니에 있는 사탕을 도착하자마자 순식간에 모두 먹을 수 있다.

수빈이가 1만큼 움직일 때 시간도 1만큼 지나가며, 수빈이는 위쪽 또는 오른쪽으로만 움직일 수 있다.

수빈이가 먹을 수 있는 사탕의 최대 개수를 구하는 프로그램을 작성하시오.

예제 입력/출력 1

입력

3 15

1 1

3 1

1 6

출력

24

김시온 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5      static int n, m;
6      static boolean[][] a;
7      static long[][] c;
8
9      // dp(x, y) = (x, y)에서 시작해서 최대한 먹을 수 있는 사탕의 양
10     static long dp(int x, int y) {
11         if (c[x][y] != -1) {
12             return c[x][y];
13         }
14         long max = 0;
15         if (x + 1 <= 300) {
16             max = Math.max(max, dp(x + 1, y));
17         }
18         if (y + 1 <= 300) {
19             max = Math.max(max, dp(x, y + 1));
20         }
21         // x + 1, y + 1방향으로만 이동 가능하기 때문에 현재 시간 t = x + y
22         long candy = a[x][y] ? Math.max(m - (x + y), 0) : 0;
23         return c[x][y] = candy + max;
24     }
25
26     public static void main(String[] args) throws IOException {
27         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
28         StringTokenizer st = new StringTokenizer(br.readLine());
29         n = Integer.parseInt(st.nextToken());
```

김시온 소스 코드 2

```
30     m = Integer.parseInt(st.nextToken());
31
32     a = new boolean[301][301];
33     for (int i = 0; i < n; i++) {
34         st = new StringTokenizer(br.readLine());
35         int x = Integer.parseInt(st.nextToken());
36         int y = Integer.parseInt(st.nextToken());
37         a[x][y] = true;
38     }
39
40     // 메모이제이션은 문제의 정답이 될 수 없는 값으로 초기화해야 한다.
41     c = new long[301][301];
42     for (int x = 0; x <= 300; x++) {
43         Arrays.fill(c[x], -1);
44     }
45
46     System.out.println(dp(0, 0));
47 }
48 }
```


원찬혁 소스 코드 1

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Main {
5      static int n, m;
6      static int dp[][];
7      static int ary[][];
8      public static void main(String[] args) throws Exception {
9          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
10         StringTokenizer st = new StringTokenizer(br.readLine());
11         n = Integer.parseInt(st.nextToken());
12         m = Integer.parseInt(st.nextToken());
13         dp = new int[301][301];
14         ary = new int[301][301];
15         for(int i = 0; i <= 300; i++) {
16             for(int j = 0; j <= 300; j++)
17                 dp[i][j] = -1;
18         }
19         for(int i = 0; i < n; i++) {
20             st = new StringTokenizer(br.readLine());
21             ary[Integer.parseInt(st.nextToken())]
22                 ↳ [Integer.parseInt(st.nextToken())] = 1;
23         }
24         System.out.println(sol(300, 300));
25     }
26     static int sol(int x, int y) {
27         if(x == 0 && y == 0) return 0;
28         if(dp[x][y] >= 0) return dp[x][y];
29         dp[x][y] = 0;
```

원찬혁 소스 코드 2

```
29         if(x > 0)
30             dp[x][y] = Math.max(dp[x][y], sol(x - 1, y));
31         if(y > 0)
32             dp[x][y] = Math.max(dp[x][y], sol(x, y - 1));
33         if(ary[x][y] == 1)
34             dp[x][y] += Math.max(0, m - (x + y));
35         return dp[x][y];
36     }
37 }
```

손현준 소스 코드 (BottomUp) 1

```
1  package baekjoon;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.util.Arrays;
7  import java.util.HashMap;
8  import java.util.StringTokenizer;
9
10 public class P14585_BottomUp {
11     static class Candy {
12         long c; // 누적 사탕 개수
13         int t; // 걸린 시간
14
15         public Candy(long c, int t) {
16             this.c = c;
17             this.t = t;
18         }
19     }
20
21     static class Point {
22         int x, y;
23         Point(int x, int y) {
24             this.x = x;
25             this.y = y;
26         }
27     }
28 }
```

손현준 소스 코드 (BottomUp) 2

```
29 public static void main(String[] args) throws IOException {
30     BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
31     StringTokenizer st;
32
33     st = new StringTokenizer(br.readLine(), " ");
34     int N = Integer.parseInt(st.nextToken());
35     int M = Integer.parseInt(st.nextToken());
36
37     Point[] points = new Point[N];
38     HashMap<Integer, Candy> baskets = new HashMap<>(); // 해당 정점까지 먹은 사탕
    ↪ 최대값
39
40
41     long maxCandy = 0;
42     for (int n = 0; n < N; n++) {
43         st = new StringTokenizer(br.readLine(), " ");
44         int xIn = Integer.parseInt(st.nextToken());
45         int yIn = Integer.parseInt(st.nextToken());
46         points[n] = new Point(xIn, yIn);
47
48         // 좌표 (0, 0) 에서 바로 가는 경우에 대한 계산을 하고 저장
49         int time = xIn + yIn;
50         long candy = (M - time <= 0) ? 0L : M - time;
51         baskets.put(getKey(xIn, yIn), new Candy(candy, time));
52         maxCandy = Math.max(maxCandy, candy);
53         // System.out.println("(" + xIn + ", " + yIn + ") - " + candy);
54     }
55 }
```

손현준 소스 코드 (BottomUp) 3

```
56     Arrays.sort(points, (o1, o2) -> {
57         if (o1.x != o2.x) return Integer.compare(o1.x, o2.x);
58         return Integer.compare(o1.y, o2.y);
59     });
60
61     for (int i = 0; i < N; i++) {
62         int xFrom = points[i].x;
63         int yFrom = points[i].y;
64
65         // 존재하지 않는 좌표라면 continue
66         int keyFrom = getKey(xFrom, yFrom);
67         Candy from = baskets.get(keyFrom);
68         if (from == null) continue;
69
70         for (int j = i + 1; j < N; j++) {
71             int xTo = points[j].x;
72             int yTo = points[j].y;
73
74             // 오른쪽 위에 있는 점만 이동 가능
75             if (xTo < xFrom || yTo < yFrom) continue;
76
77             // 존재하지 않는 좌표라면 continue
78             int keyTo = getKey(xTo, yTo);
79             Candy next = baskets.get(keyTo);
80             if (next == null) continue;
81
82             int nextT = from.t + (xTo - xFrom) + (yTo - yFrom);
83             long nextC = M - nextT;
```

손현준 소스 코드 (BottomUp) 4

```
84         if (nextC <= 0) continue; // 사탕이 모두 녹아서 사라졌다면 continue
85
86         nextC += from.c; // 이전 사탕 개수 더해주기
87         if (next.c < nextC) {
88             baskets.put(keyTo, new Candy(nextC, nextT));
89             maxCandy = Math.max(maxCandy, nextC);
90             // System.out.println("(" + xFrom + ", " + yFrom + ") > (" + xTo +
↪      " , " + yTo + ") - " + nextC);
91         }
92     }
93 }
94 System.out.println(maxCandy);
95 }
96
97 static int getKey(int x, int y) {
98     return (x << 9) | y;
99 }
100 }
```

손현준 소스 코드 (DFS) 1

```
1  package baekjoon;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.util.StringTokenizer;
7
8  public class P14585_DFS {
9      static int N, M;
10     static Basket[] basket;
11     static long dp[];
12
13     static class Basket {
14         int x, y;
15
16         Basket(int x, int y) {
17             this.x = x;
18             this.y = y;
19         }
20     }
21
22     public static void main(String[] args) throws IOException {
23         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
24         StringTokenizer st;
25
26         st = new StringTokenizer(br.readLine(), " ");
27         N = Integer.parseInt(st.nextToken());
28         M = Integer.parseInt(st.nextToken());
```

손현준 소스 코드 (DFS) 2

```
29
30     basket = new Basket[N + 1];
31     basket[0] = new Basket(0, 0);
32     for (int n = 1; n <= N; n++) {
33         st = new StringTokenizer(br.readLine(), " ");
34         int x = Integer.parseInt(st.nextToken());
35         int y = Integer.parseInt(st.nextToken());
36         basket[n] = new Basket(x, y);
37     }
38
39     dp = new long[N + 1];
40     System.out.println(func(0, 0, 0));
41 }
42
43 /*
44  * 걸린 시간
45  * 마지막 방문 바구니의 인덱스
46  * 누적 사탕
47  */
48 static long func(int t, int i, long c) {
49     int nowX = basket[i].x;
50     int nowY = basket[i].y;
51
52     if (t >= M) {
53         return c;
54     }
55
56     // 다음으로 바구니를 방문하는 경우를 모두 확인
```


손현준 소스 코드 (DFS) 3

```
57         long result = c;
58         for (int n = 1; n <= N; n++) {
59             int nextX = basket[n].x;
60             int nextY = basket[n].y;
61
62             // 우측 상단으로만 이동
63             if (nextX < nowX || nextY < nowY || (nextX == nowX && nextY == nowY))
64                 ↪ continue;
65
66             // 거리 계산
67             int dist = t + (nextX - nowX) + (nextY - nowY);
68             long nCandy = c + M - dist;
69
70             if (dp[n] > nCandy) continue;
71             dp[n] = nCandy;
72
73             result = Math.max(result, func(dist, n, nCandy));
74         }
75         return result;
76     }
77 }
```

손현준 소스 코드 (TopDown) 1

```
1 package baekjoon;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.util.StringTokenizer;
7
8 public class P14585_TopDown {
9     static int N, M;
10    static int[][] basket;
11    static int[][] dp;
12
13    public static void main(String[] args) throws IOException {
14        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
15        StringTokenizer st;
16
17        st = new StringTokenizer(br.readLine());
18        N = Integer.parseInt(st.nextToken());
19        M = Integer.parseInt(st.nextToken());
20
21        basket = new int[301][301];
22        for (int n = 0; n < N; n++) {
23            st = new StringTokenizer(br.readLine(), " ");
24            int x = Integer.parseInt(st.nextToken());
25            int y = Integer.parseInt(st.nextToken());
26            basket[x][y] = 1; // 해당 좌표에 바구니가 있는지 기록
27        }
28    }
```

손현준 소스 코드 (TopDown) 2

```
29         dp = new int[301][301];
30         for (int x = 0; x <= 300; x++) {
31             for (int y = 0; y <= 300; y++) {
32                 dp[x][y] = -1;
33             }
34         }
35
36         System.out.println(func(300, 300));
37     }
38
39     static int func(int x, int y) {
40         if (x == 0 && y == 0) return 0;
41         if (dp[x][y] >= 0) return dp[x][y];
42
43         if (x > 0) dp[x][y] = Math.max(dp[x][y], func(x - 1, y));
44         if (y > 0) dp[x][y] = Math.max(dp[x][y], func(x, y - 1));
45         if (basket[x][y] == 1) { // 바구니가 존재할때, 바구니에 남은 사탕 수 더해줌
46             int t = x + y;
47             dp[x][y] += (M < t) ? 0 : M - t;
48         }
49
50         return dp[x][y];
51     }
52 }
```

김준형 소스 코드 (BackTracking) 1

```
1 package algo_test;
2
3 import java.io.*;
4 import java.util.*;
5
6 public class Main_BackTracking {
7     static class Cord{
8         int y;
9         int x;
10        public Cord(int y, int x) {
11            this.y = y;
12            this.x = x;
13        }
14    }
15    static int n,m,ans = -1;
16    static Cord[] candies;
17    static int cache[][];
18    public static void main(String[] args) throws IOException {
19        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
20        StringTokenizer st = new StringTokenizer(br.readLine());
21
22        n = Integer.parseInt(st.nextToken());
23        m = Integer.parseInt(st.nextToken());
24        candies = new Cord[n];
25        cache = new int[301][301];
26        for(int i=0; i<301; i++) {
27            Arrays.fill(cache[i], -1);
28        }
```

김준형 소스 코드 (BackTracking) 2

```
29         for(int i=0; i<n; i++) {
30             st = new StringTokenizer(br.readLine());
31             int x = Integer.parseInt(st.nextToken());
32             int y = Integer.parseInt(st.nextToken());
33             candies[i] = new Cord(y,x);
34         }
35
36         backTracking(0,0,0,0);
37         System.out.print(ans);
38     }
39
40     static void backTracking(int y, int x, int ate, int t) {
41         if(cache[y][x] > ate || m<t) {
42             return;
43         }
44
45         cache[y][x] = ate;
46         ans = Math.max(ans, ate);
47
48         for(int i=0; i<n; i++) {
49             Cord next = candies[i];
50             int ny = next.y; int nx = next.x;
51             if(ny>y && nx>=x || nx>x && ny>=y) {
52                 int time = getDistance(ny, nx, y, x);
53                 backTracking(ny, nx, ate+m-(t+time), t+time);
54             }
55         }
56     }
```

김준형 소스 코드 (BackTracking) 3

```
57
58     static int getDistance(int y2,int x2, int y1, int x1) {
59         return Math.abs(y2-y1)+Math.abs(x2-x1);
60     }
61
62 }
```

김준형 소스 코드 (DP) 1

```
1  package algo_test;
2
3  import java.io.*;
4  import java.util.*;
5
6  public class Main_DP {
7      static class Cord{
8          int y;
9          int x;
10         public Cord(int y, int x) {
11             this.y = y;
12             this.x = x;
13         }
14     }
15     static int n,m;
16     static Cord[] candies;
17     static int dp[][];
18     public static void main(String[] args) throws IOException {
19         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
20         StringTokenizer st = new StringTokenizer(br.readLine());
21
22         n = Integer.parseInt(st.nextToken());
23         m = Integer.parseInt(st.nextToken());
24         candies = new Cord[n];
25         dp = new int[301][301];
26         for(int i=0; i<n; i++) {
27             st = new StringTokenizer(br.readLine());
28             int x = Integer.parseInt(st.nextToken());
```

김준형 소스 코드 (DP) 2

```
29         int y = Integer.parseInt(st.nextToken());
30         candies[i] = new Cord(y,x);
31     }
32
33     System.out.println(backTracking(0, 0, 0));
34 }
35
36 static int backTracking(int y, int x, int t) {
37     if(dp[y][x]>0 || m<t) {
38         return dp[y][x];
39     }
40
41     for(int i=0; i<n; i++) {
42         Cord next = candies[i];
43         int ny = next.y; int nx = next.x;
44         if(ny>y && nx>=x || nx>x && ny>=y) {
45             int time = getDistance(ny, nx, y, x);
46             dp[y][x] = Math.max(dp[y][x], backTracking(ny,nx,t+time));
47         }
48     }
49
50     return dp[y][x]+=(t==0?0:(m-t));
51 }
52
53 static int getDistance(int y2,int x2, int y1, int x1) {
54     return Math.abs(y2-y1)+Math.abs(x2-x1);
55 }
56 }
```


서민종 소스 코드 1

```
1 package BOJ14585;
2
3 import java.util.*;
4 import java.io.*;
5
6 public class Main {
7     static int N, M;
8     static int[][] map;
9     public static void main(String[] args) throws IOException {
10         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
11         BufferedWriter bw = new BufferedWriter(new
            ↳ OutputStreamWriter(System.out));
12         StringTokenizer st = new StringTokenizer(br.readLine());
13         N = Integer.parseInt(st.nextToken());
14         M = Integer.parseInt(st.nextToken());
15         map = new int[301][301];
16         for(int i=0; i<N; i++) {
17             st = new StringTokenizer(br.readLine());
18             int x = Integer.parseInt(st.nextToken());
19             int y = Integer.parseInt(st.nextToken());
20             map[x][y] = M;
21         }
22         int[][] dp = new int[301][301];
23         for(int x=0; x<=300; x++) {
24             for(int y=0; y<=300; y++) {
25                 if(x == 0 && y == 0) {
26                     continue;
27                 }
28                 int candy = (map[x][y] > x + y) ? M - x - y : 0;
```

서민종 소스 코드 2

```
29         if(x == 0) {
30             dp[x][y] = dp[x][y-1] + candy;
31         } else if(y == 0) {
32             dp[x][y] = dp[x-1][y] + candy;
33         } else {
34             dp[x][y] = Math.max(dp[x-1][y], dp[x][y-1]) + candy;
35         }
36     }
37 }
38 int answer = dp[300][300];
39 bw.write(answer + "\n");
40 bw.flush();
41 }
42 }
```

임건애 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5
6      static int N, M;
7      static int[][] map, memo;
8      static int MAX = 300;
9
10     public static void main(String[] args) throws IOException {
11         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
12         StringTokenizer st = new StringTokenizer(br.readLine());
13
14         N = Integer.parseInt(st.nextToken());
15         M = Integer.parseInt(st.nextToken());
16
17         map = new int[MAX + 1][MAX + 1];
18         for (int i = 0; i < N; i++) {
19             st = new StringTokenizer(br.readLine());
20             int x = Integer.parseInt(st.nextToken());
21             int y = Integer.parseInt(st.nextToken());
22
23             map[x][y] = 1;
24         }
25
26         memo = new int[MAX + 1][MAX + 1];
27         for (int i = 0; i <= MAX; i++) {
28             Arrays.fill(memo[i], -1);
29         }
30     }
31 }
```

임건애 소스 코드 2

```
30
31     solve(MAX, MAX);
32
33     System.out.println(memo[MAX][MAX]);
34
35 }
36
37 static int solve(int i, int j) {
38     if (i < 0 || j < 0) return 0;
39
40     if (memo[i][j] != -1) return memo[i][j];
41
42     int candy = 0;
43     if (map[i][j] == 1) {
44         candy = Math.max(0, M - (i + j));
45     }
46
47     return memo[i][j] = Math.max(solve(i - 1, j), solve(i, j - 1)) + candy;
48 }
49
50 }
```

임건애 소스 코드 (BottomUp) 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5
6      public static void main(String[] args) throws IOException {
7          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
8          StringTokenizer st = new StringTokenizer(br.readLine());
9
10         int N = Integer.parseInt(st.nextToken());
11         int M = Integer.parseInt(st.nextToken());
12
13         int[][] map = new int[301][301];
14
15         for (int i = 0; i < N; i++) {
16             st = new StringTokenizer(br.readLine());
17             int x = Integer.parseInt(st.nextToken());
18             int y = Integer.parseInt(st.nextToken());
19
20             map[x][y] = 1;
21         }
22
23         int[][] dp = new int[301][301];
24         int result = 0;
25
26         for (int i = 0; i < 301; i++) {
27             for (int j = 0; j < 301; j++) {
28                 if (i == 0 && j == 0)
```

임건애 소스 코드 (BottomUp) 2

```
29         continue;
30     if (i == 0) {
31         dp[i][j] = dp[i][j - 1];
32     } else if (j == 0) {
33         dp[i][j] = dp[i - 1][j];
34     } else {
35         dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
36     }
37
38     if (map[i][j] == 1) {
39         int candy = M - (i + j);
40         if (candy > 0) {
41             dp[i][j] += candy;
42         }
43     }
44     result = Math.max(result, dp[i][j]);
45 }
46 }
47
48 System.out.println(result);
49 }
50
51 }
```

사건은 다가와 (Easy)(32198)

문제

민정이는 현재 수직선의 원점, 즉 위치 0에 있다. 민정이는 매 시점 수직선 위에서 왼쪽 또는 오른쪽으로 초당 1의 속도로 이동할 수 있고, 정지해 있을 수도 있다.

민정이는 앞으로 사건이 $N(1 \leq N \leq 100)$ 번 발생할 것을 알고 있다. 각 사건은 정수 시각 $T(1 \leq T \leq 1000)$ 에 발생하며, 그때 민정이가 위치 A 와 위치 $B(-1000 \leq A < B \leq 1000)$ 에 대해 열린 구간 (A, B) 안에 있으면 카리나의 body bang을 맞게 된다. 위치 A 와 위치 B 는 안전하다는 점에 유의하라.

각 사건의 T 는 모두 다르다.

민정이는 카리나의 body bang을 맞지 않기 위해 적절히 움직이려 한다. 민정이가 body bang을 피할 수 있는지 판별하고, 피할 수 있다면 이동 거리의 최솟값을 구하여라.

예제 입력/출력 1

입력

3
10 -3 7
20 -8 2
25 3 9

출력

8

김시온 소스 코드 1

```
1  import java.util.*;
2  import java.io.*;
3
4  public class Main {
5      static int n;
6      static int[][] a;
7      static int[][] c;
8
9      static int dp(int t, int x) {
10         if (t == 1001) {
11             return 0;
12         }
13         if (c[t][x] != -1) {
14             return c[t][x];
15         }
16
17         int l = a[t][0];
18         int r = a[t][1];
19         if (l < x && x < r) {
20             // 정답이 될 수 없는 값을 반환
21             return c[t][x] = (int) 1e9;
22         }
23
24         // 1. 가만히 있기
25         int min = dp(t + 1, x);
26         // 2. 왼쪽으로 이동
27         if (x - 1 >= 0) {
28             min = Math.min(min, 1 + dp(t + 1, x - 1));
29         }
30     }
31 }
```

김시온 소스 코드 2

```
30         // 3. 오른쪽으로 이동
31         if (x + 1 <= 2000) {
32             min = Math.min(min, 1 + dp(t + 1, x + 1));
33         }
34
35         return c[t][x] = min;
36     }
37
38     public static void main(String[] args) throws IOException {
39         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
40         n = Integer.parseInt(br.readLine());
41
42         a = new int[1001][2];
43         for (int i = 0; i < n; i++) {
44             StringTokenizer st = new StringTokenizer(br.readLine());
45             int t = Integer.parseInt(st.nextToken());
46             int l = Integer.parseInt(st.nextToken());
47             int r = Integer.parseInt(st.nextToken());
48
49             a[t][0] = l + 1000;
50             a[t][1] = r + 1000;
51         }
52
53         c = new int[1001][2001];
54         for (int i = 0; i <= 1000; i++) {
55             Arrays.fill(c[i], -1);
56         }
57     }
```

김시온 소스 코드 3

```
58         int r = dp(0, 1000);  
59         if (r == (int) 1e9) {  
60             System.out.println(-1);  
61         } else {  
62             System.out.println(r);  
63         }  
64     }  
65 }
```

원찬혁 소스 코드 1

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Main {
5      static final int inf = 1000000000;
6      static int n;
7      static int dp[][];
8      public static void main(String[] args) throws IOException {
9          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
10         StringTokenizer st;
11         StringBuilder sb = new StringBuilder();
12         n = Integer.parseInt(br.readLine());
13         int t = 0, a, b, res = inf, maxt = 0;
14         dp = new int[1001][2001];
15         for(int i = 0; i <= 1000; i++)
16             for(int j = 0; j <= 2000; j++)
17                 dp[i][j] = -1;
18         while(n-- > 0) {
19             st = new StringTokenizer(br.readLine());
20             t = Integer.parseInt(st.nextToken());
21             a = Integer.parseInt(st.nextToken()) + 1000;
22             b = Integer.parseInt(st.nextToken()) + 1000;
23             maxt = Math.max(maxt, t);
24             for(int i = a + 1; i < b; i++)
25                 dp[t][i] = inf;
26         }
27         for(int i = 0; i <= 2000; i++)
28             res = Math.min(res, sol(maxt, i));
29     }
```

원찬혁 소스 코드 2

```
30         if(res == inf) System.out.println(-1);
31         else System.out.println(res);
32     }
33
34     static int sol(int t, int loc) {
35         if(loc < 0 || loc > 2000) return inf;
36         if(t == 0) {
37             if(loc == 1000) return 0;
38             return inf; // 불가능
39         }
40         if(dp[t][loc] >= 0) return dp[t][loc];
41         // 정지, 왼쪽, 오른쪽
42         dp[t][loc] = sol(t - 1, loc);
43         dp[t][loc] = Math.min(dp[t][loc], sol(t - 1, loc - 1) + 1);
44         dp[t][loc] = Math.min(dp[t][loc], sol(t - 1, loc + 1) + 1);
45         return dp[t][loc];
46     }
47 }
```

손현준 소스 코드 1

```
1  package baekjoon;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.util.ArrayList;
7  import java.util.HashMap;
8  import java.util.List;
9  import java.util.StringTokenizer;
10
11 public class P32198 {
12     static final int INF = Integer.MAX_VALUE;
13     static int[][] dp;
14     static boolean[][] visited;
15     static HashMap<Integer, Bang> bang;
16     static int maxT;
17
18     static class Bang {
19         int a, b;
20
21         public Bang(int a, int b) {
22             this.a = a;
23             this.b = b;
24         }
25     }
26
27     public static void main(String[] args) throws IOException {
28         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
29         StringTokenizer st;
```

손현준 소스 코드 2

```
30
31     int N = Integer.parseInt(br.readLine());
32     bang = new HashMap<>();
33     maxT = 0;
34     for (int n = 0; n < N; n++) {
35         st = new StringTokenizer(br.readLine());
36         int T = Integer.parseInt(st.nextToken());
37         int A = Integer.parseInt(st.nextToken()) + 1000;
38         int B = Integer.parseInt(st.nextToken()) + 1000;
39         maxT = Math.max(maxT, T);
40         bang.put(T, new Bang(A, B));
41     }
42
43     dp = new int[maxT + 1][2001]; // dp[시간][위치] > 위치index: 0~999(-1000~-1)
44     ↪ - 1000(0) - 1001~2000(1~1000)
45     visited = new boolean[maxT + 1][2001];
46     // dp 배열 초기화
47     for (int i = 0; i <= maxT; i++) {
48         for (int j = 0; j <= 2000; j++) {
49             dp[i][j] = INF;
50         }
51     }
52
53     int result = solve(0, 1000);
54     System.out.println(result == INF ? -1 : result);
55 }
56 /*
```

손현준 소스 코드 3

```
57     * t: 시간
58     * p: 위치
59     */
60     static int solve(int t, int p) {
61         if (t == maxT) return 0;
62
63         if (visited[t][p]) return dp[t][p];
64         visited[t][p] = true;
65
66         int result = INF;
67         Bang nextB = bang.get(t + 1);
68         // 왼쪽 오른쪽 끝을 넘어가지 않도록 처리
69         if (p > 0) { // 왼쪽으로 갈 수 있을 때
70             int nextP = p - 1;
71             if (nextB == null || (nextP <= nextB.a || nextP >= nextB.b)) {
72                 int temp = solve(t + 1, nextP);
73                 if (temp != INF) result = Math.min(result, temp + 1);
74             }
75         }
76         if (p < 2000) { // 오른쪽으로 갈 수 있을 때
77             int nextP = p + 1;
78             if (nextB == null || (nextP <= nextB.a || nextP >= nextB.b)) {
79                 int temp = solve(t + 1, nextP);
80                 if (temp != INF) result = Math.min(result, temp + 1);
81             }
82         }
83         // 움직이지 않을 때
84         if (nextB == null || (p <= nextB.a || p >= nextB.b)) {
```


손현준 소스 코드 4

```
85         int temp = solve(t + 1, p);
86         if (temp != INF) result = Math.min(result, temp);
87     }
88
89     return dp[t][p] = result;
90 }
91 }
```

김준형 소스 코드 1

```
1  import java.io.*;
2  import java.util.*;
3
4  public class Main {
5      static class Bang implements Comparable<Bang>{
6          int t;
7          int a;
8          int b;
9          public Bang(int t, int a, int b) {
10              this.t = t;
11              this.a = a;
12              this.b = b;
13          }
14
15          @Override
16          public int compareTo(Bang b) {
17              return Integer.compare(this.t, b.t);
18          }
19      }
20
21      static Bang[] bangs;
22      static int[][] dp = new int[100][2001];
23      static int n;
24      public static void main(String[] args) throws IOException {
25          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
26          StringTokenizer st;
27          n = Integer.parseInt(br.readLine());
28          bangs = new Bang[n];
29      }
```

김준형 소스 코드 2

```
30     for(int i=0; i<n; i++) {
31         st = new StringTokenizer(br.readLine());
32         int t = Integer.parseInt(st.nextToken());
33         int a = Integer.parseInt(st.nextToken())+1000;
34         int b = Integer.parseInt(st.nextToken())+1000;
35         bangs[i] = new Bang(t,a,b);
36     }
37     Arrays.sort(bangs);
38
39     for(int i=0; i<100; i++) {
40         Arrays.fill(dp[i], -1);
41     }
42
43     int ans = go(0,1000);
44     System.out.print(ans==20000001?-1:ans);
45 }
46
47 static int go(int ac, int cur) {
48     if(ac>=n) {
49         return 0;
50     }
51
52     if(dp[ac][cur]!=-1) { // 갱신 했다는 것
53         return dp[ac][cur];
54     }
55
56     int nt = bangs[ac].t; int bt = ac==0?0:bangs[ac-1].t;
57     int a = bangs[ac].a; int b = bangs[ac].b;
```

김준형 소스 코드 3

```
58
59     int minV = 2000001;
60     for(int i=cur-(nt-bt); i<=cur+(nt-bt); i++) {
61         if(i<=a || i>=b) {
62             minV = Math.min(minV, go(ac+1,i)+Math.abs(cur-i));
63         }
64     }
65     dp[ac][cur] = minV;
66     return dp[ac][cur];
67 }
68 }
```

정의찬 소스 코드 1

```
1  import java.io.*;
2  import java.util.*;
3
4  class Main {
5      static final int IMPOSSIBLE = 100000000;
6
7      public static void main(String[] args) throws NumberFormatException,
8          ↳ IOException {
9          BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
10         StringBuilder sb = new StringBuilder();
11         StringTokenizer st;
12
13         int n = Integer.parseInt(br.readLine());
14         int[][] dp = new int[n + 1][2001];
15         for (int i = 0; i <= n; i++) {
16             Arrays.fill(dp[i], IMPOSSIBLE);
17         }
18         dp[0][1000] = 0; // 첫위치:1000
19
20         PriorityQueue<int[]> pq = new PriorityQueue<>((o1, o2) ->
21             ↳ Integer.compare(o1[0], o2[0]));
22
23         for (int i = 0; i < n; i++) {
24             st = new StringTokenizer(br.readLine());
25             int[] input = new int[3];
26             input[0] = Integer.parseInt(st.nextToken());
27             input[1] = Integer.parseInt(st.nextToken()) + 1000;
28             input[2] = Integer.parseInt(st.nextToken()) + 1000;
29             pq.add(input);
```

정의찬 소스 코드 2

```
28     }
29
30     int prevT = 0;
31     for (int i = 0; i < n; i++) {
32         int[] c = pq.poll();
33         int currentT = c[0];
34         int t = currentT - prevT;
35         int l = c[1];
36         int r = c[2];
37
38         for (int j = 0; j <= 2000; j++) {
39             if (dp[i][j] == IMPOSSIBLE)
40                 continue;
41             for (int k = -t; k <= t; k++) {
42                 if (j + k <= l || r <= j + k)
43                     dp[i + 1][j + k] = Math.min(dp[i][j] + Math.abs(k), dp[i
44                                     ↪ + 1][j + k]);
45             }
46             prevT = currentT;
47         }
48         int ans = Integer.MAX_VALUE;
49         for (int i = 0; i <= 2000; i++) {
50             if (dp[n][i] != IMPOSSIBLE) {
51                 ans = Math.min(ans, dp[n][i]);
52             }
53         }
54     }
```

정의찬 소스 코드 3

```
55         System.out.println(ans == Integer.MAX_VALUE ? -1 : ans);
56
57     }
58 }
```